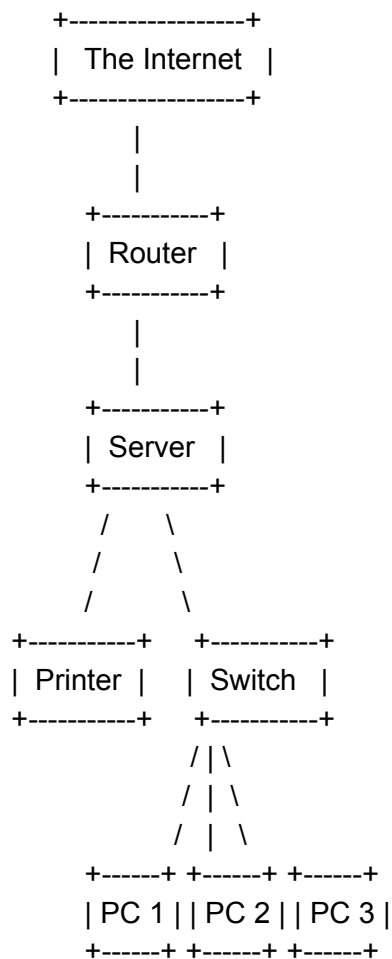


Introduction to IP Addresses

- IP (Internet Protocol) addresses are unique numerical labels assigned to devices on a network.
- They enable communication between different machines, servers, and services across the internet.
- Two primary versions:
 - IPv4
 - IPv6
- Two primary categories:
 - Public
 - Private



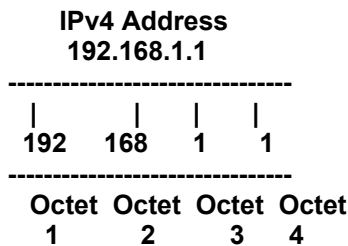
What is IPv4?

- IPv4 (Internet Protocol Version 4) is the most widely used addressing system.
- 32-bit address format (e.g., 192.168.1.1)
- Total addresses available: ~4.3 billion
- Uses: Traditional networking, web servers, and most current internet devices.
- Challenges: Limited IPs, fragmentation, and security concerns.

Example IPv4 Address

192.168.1.1

IPv4 Structure Diagram



Binary Representation of IPv4

An IPv4 address is a **32-bit** number.

192 .168 .1 .1
11000000 10101000 00000001 00000001

Each section is called an octet:

- 1 octet = 8 bits
- 4 octets × 8 bits = 32 bits total

IPv4 Working Concept :

Laptop/Phone

|

[Router]

|

Public IP

|

Internet

|

Web Server

Why IPv4 became popular:

IPv4 was simple, lightweight, and easy to implement in early internet systems.

Why IPv4 is running out:

Maximum possible IPv4 addresses:
 $2^{32} \approx 4.3$ billion

Because billions of:

- phones
- laptops
- servers
- IoT devices

now exist, IPv4 addresses became insufficient.

Real-World Usage

IPv4 is still heavily used in:

- Home WiFi networks
- Corporate networks
- Cloud VMs
- Traditional servers
- Routers and switches
- Most existing internet infrastructure

However, IPv6 adoption is increasing because IPv4 address space is limited.

What is IPv6?

- IPv6 (Internet Protocol Version 6) is the next-generation IP standard.
- 128-bit address format -
`2001:0db8:85a3:0000:0000:8a2e:0370:7334`
- Total addresses available: 340 undecillion (virtually unlimited).
- Designed for: IoT, mobile networks, and future scalability.
- Key benefits: Larger address space, better security, and improved routing efficiency.

IPv6 Address Example:

2001:0db8:85a3:0000:0000:8a2e:0370:7334

IPv6 Binary Size

IPv6 uses a **128-bit** address space.

Comparison:

IPv4 = 32 bits

IPv6 = 128 bits

Total possible IPv6 addresses:

$2^{128} \approx 3.4 \times 10^{38}$

This is approximately:

340,000,000,000,000,000,000,000,000,000,000

which is called **340 undecillion** addresses.

IPv6 Address Representation

IPv6 uses:

- Hexadecimal numbers
- Colon (:) separators
- 8 groups
- Each group contains 16 bits

Example:

2001 : 0db8 : 85a3 : 0000 : 0000 : 8a2e : 0370 : 7334

Why IPv6 Was Introduced

IPv4 addresses were becoming insufficient because of:

- Smartphones
- IoT devices
- Cloud computing
- Billions of internet-connected devices

IPv6 solves this by providing an enormous address space.

IPv4 vs IPv6 :

IPv4

Uses 32-bit addresses.

Example:

192.168.1.1

Total addresses:

≈ 4.3 billion

IPv6

Uses 128-bit addresses.

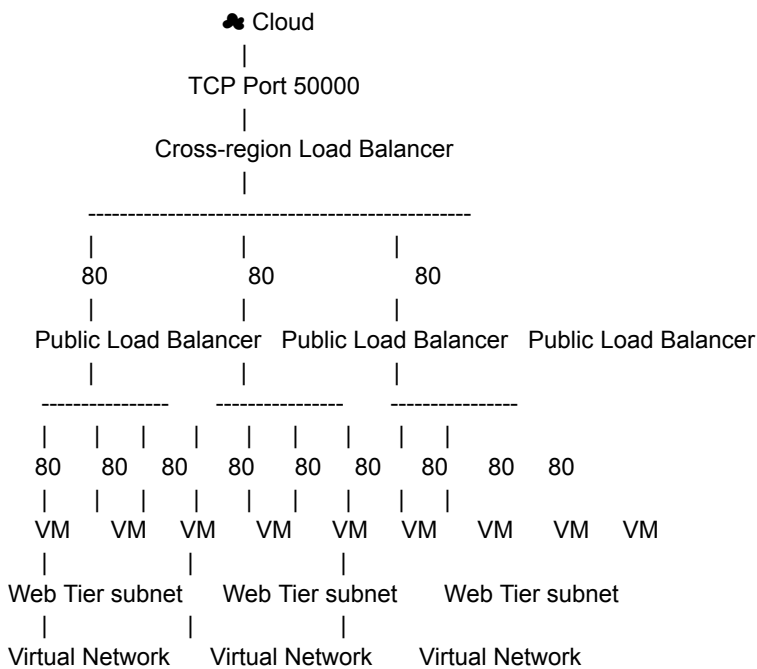
Example:

2001:0db8:85a3:0000:0000:8a2e:0370:7334

Provides a massive number of addresses for modern internet devices.

The Role of IPs in System Design

- **Scalability:** Helps in designing distributed, multi-region, and cloud-based architectures.
- **Security:** Enables firewall rules, VPNs, and private networking.
- **Load Balancing:** Uses IP-based traffic distribution (*e.g., Round-robin DNS, Anycast IPs*).
- **Cloud Networking:** Public, private, and hybrid cloud IP management (*AWS, GCP, Azure*).
- **Microservices & Containers:** Use internal private IPs for communication.



Flow Explanation

1. Users connect through the **Cloud endpoint**.
2. Traffic enters via **TCP Port 50000**.
3. A **Cross-region Load Balancer** distributes traffic across regions.
4. Each region has a **Public Load Balancer**.
5. The load balancer distributes requests to multiple **VMs** on port **80**.
6. VMs exist inside a **Web Tier subnet**.
7. Subnets are part of a **Virtual Network (VNet)**.

This architecture demonstrates:

- Horizontal scaling
- High availability
- Multi-region deployment
- Traffic distribution using IP networking
- Cloud-native infrastructure design

Feature	IPv4	IPv6
Address Size	32-bit	128-bit
Address Capacity	~4.3 billion	~340 undecillion
NAT Requirement	Common	Mostly unnecessary
Security	Optional	Better built-in support
Routing Efficiency	Moderate	Improved
IoT Scalability	Limited	Excellent

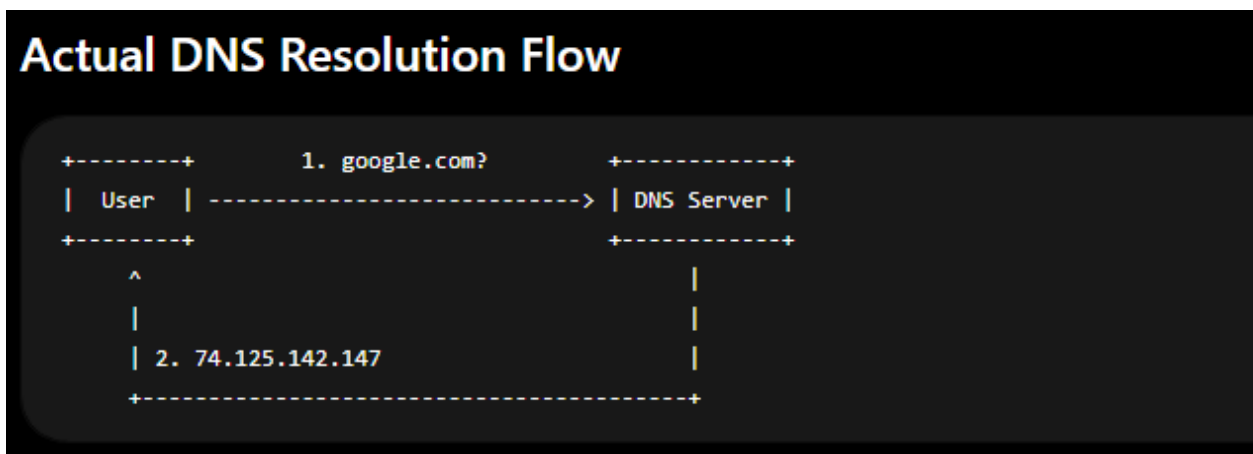
Introduction to DNS

What is DNS?

- The "phonebook of the internet."
- Translates human-readable domain names (e.g., google.com) into IP

Why is DNS important?

- Enables accessibility of websites.



DNS (Domain Name System)

DNS is a distributed system that converts domain names into IP addresses. It allows humans to use easy names like [google.com](#) instead of numeric IPs.

Domain Name

A human-readable website name such as [google.com](#). Users type domain names because they are easier to remember.

IP Address

The actual network address of a server on the internet. Computers use IP addresses to communicate with each other.

DNS Server

A specialized server that stores DNS records.

It answers queries and returns the IP address corresponding to a domain name.

A user enters:

`https://logistics.abc.com`



DNS translates it into:

`logistics.abc.com`



`20.40.110.55`



Then the browser sends requests to:

`20.40.110.55`



The user only remembers **logistics.abc.com**, while DNS finds the actual IP address.

DNS (Domain Name System) is a distributed naming system that translates human-readable domain names into IP addresses, enabling users to access websites and services without remembering numerical IP addresses.

Think of DNS exactly like your phone contacts:

You remember:

`"Mrinmoy"`



Phone stores:

`+91XXXXXXXXXX`

Similarly:

You remember:

`google.com`



DNS stores:

`142.250.193.78`

Just as your phone converts a name into a phone number, DNS converts a domain name into an IP address.

This is why DNS is called **"The Phonebook of the Internet."**

DNS Caching and Performance Optimization

Why caching matters?

- Reduces latency.
- Reduces load on DNS servers.

Where caching occurs?

- Browser cache (stores previous lookups).
- OS cache (/etc/hosts, Windows DNS cache).
- Recursive Resolver cache (ISP level).

Time-To-Live (TTL)

TTL determines how long a cached record is valid.

DNS Caching Flow

Request #1

User

|

| logistics.abc.com ?

v

DNS Server

|

| 20.40.110.55

v

User

Result stored in cache

Request #2

User

|

| logistics.abc.com ?

v

Browser Cache

|

| 20.40.110.55

v

User

DNS server is NOT contacted

Browser Cache

The browser stores recently resolved domain names locally.
Future requests can use the cached IP without contacting a DNS server.

OS Cache

The operating system maintains a DNS cache shared by applications.
This avoids repeated DNS lookups from the same machine.

Recursive Resolver Cache

The ISP or public DNS resolver stores previous DNS responses.
Many users benefit from the same cached records.

TTL (Time-To-Live)

TTL specifies how long a DNS record can remain in cache.
Once TTL expires, a fresh lookup must be performed.

Latency

Latency is the time taken for a request to travel and receive a response.
Lower latency means faster website loading and better user experience

Suppose a user opens:

`https://tracking.abc.com`



First Visit

`tracking.abc.com`



↓

DNS Lookup

↓

`20.40.110.55`

Time taken: 50 ms

Second Visit

`tracking.abc.com`



↓

Browser Cache

↓

`20.40.110.55`

Time taken: 1–2 ms

No DNS server call is required.

Importance of DNS in Large-Scale Systems

Ensuring High Availability

- Load balancing using DNS.
- Anycast DNS for faster global resolution.

DNS Failover Strategies

- Primary and secondary DNS servers.

Content Delivery Networks (CDNs) & DNS

- How DNS directs users to the nearest CDN node for fast loading.

DNS Security Risks

- DNS poisoning, cache poisoning, and DDoS attacks.

DNS Load Balancing

DNS can return different IP addresses for the same domain.
This distributes traffic across multiple servers.

Example

```
api.abc.com
|
+----> 20.1.1.10
+----> 20.1.1.11
+----> 20.1.1.12
```

Users are distributed among these servers.

Anycast DNS

Multiple DNS servers around the world share the same IP address.
The user is automatically routed to the nearest DNS server.

Example

```
India User
|
+-----> India DNS
```

```
USA User
|
+-----> USA DNS
```

```
Europe User
|
+-----> Europe DNS
```

Result:

- Faster DNS lookup
- Lower latency
- Better user experience

DNS Failover

DNS can redirect users to a backup server when the primary server fails.

Example

Normal State

```
app.abc.com
|
+-----> Server A
```

Server A Down

```
app.abc.com
|
+-----> Server B
```

Users continue using the application without noticing the failure.

CDN + DNS

DNS helps direct users to the closest CDN location.

```
User (Mumbai)
|
+----> CDN Mumbai

User (London)
|
+----> CDN London

User (New York)
|
+----> CDN New York
```

This reduces content download time.

DNS Security Risks

DNS Poisoning

An attacker tricks DNS into returning a fake IP address.

```
google.com
|
+----> Fake Server
```

User reaches the attacker's website instead of the real one.

Cache Poisoning

A malicious DNS response is inserted into a DNS cache.

```
DNS Cache

google.com
|
+----> Wrong IP
```

All users using that cache may be redirected incorrectly.

DNS DDoS Attack

Attackers flood DNS servers with massive traffic.

Millions of Requests

|

v

DNS Server

|

Overloaded

Legitimate users may be unable to resolve domains.

Logistics Example

Imagine a logistics tracking system:

tracking.abc.com

DNS Load Balancing

tracking.abc.com

|

+----> Server 1

+----> Server 2

+----> Server 3

Thousands of users can track shipments simultaneously.

DNS Failover

Server 1 Down

|

DNS automatically points users to

|

Server 2

The tracking portal remains available.

CDN

Shipment documents, POD files, and images are cached closer to users.

tracking.abc.com

|

DNS

|

Nearest CDN

|

User

Downloads become much faster.

What is the Client-Server Model?

- A computing model where clients request services and servers provide them.
- Foundation of modern web, database, and application architectures.

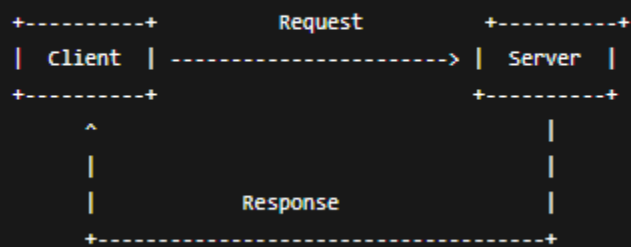
Why is it important?

- a. Enables efficient resource management.
- b. Used in web browsing, email, APIs, databases, and cloud services.

Real-world examples:

- a. Browsing a website (Browser = Client, Web Server = Server).
- b. Sending an email (Email App = Client, Mail Server = Server).

Actual Client-Server Request Flow



Imagine a shipment tracking portal.

Step 1

User opens:

```
tracking.abc.com
```

Step 2

Browser sends request:

```
Client ---> Server
```

Request:

```
<> http
GET /track?shipment=ABC123
```

Step 3

Server processes:

```
Check Database
Find Shipment
Prepare Response
```

Step 4

Server returns:

```
<> JSON
{
  "shipment": "ABC123",
  "status": "Delivered"
}
```

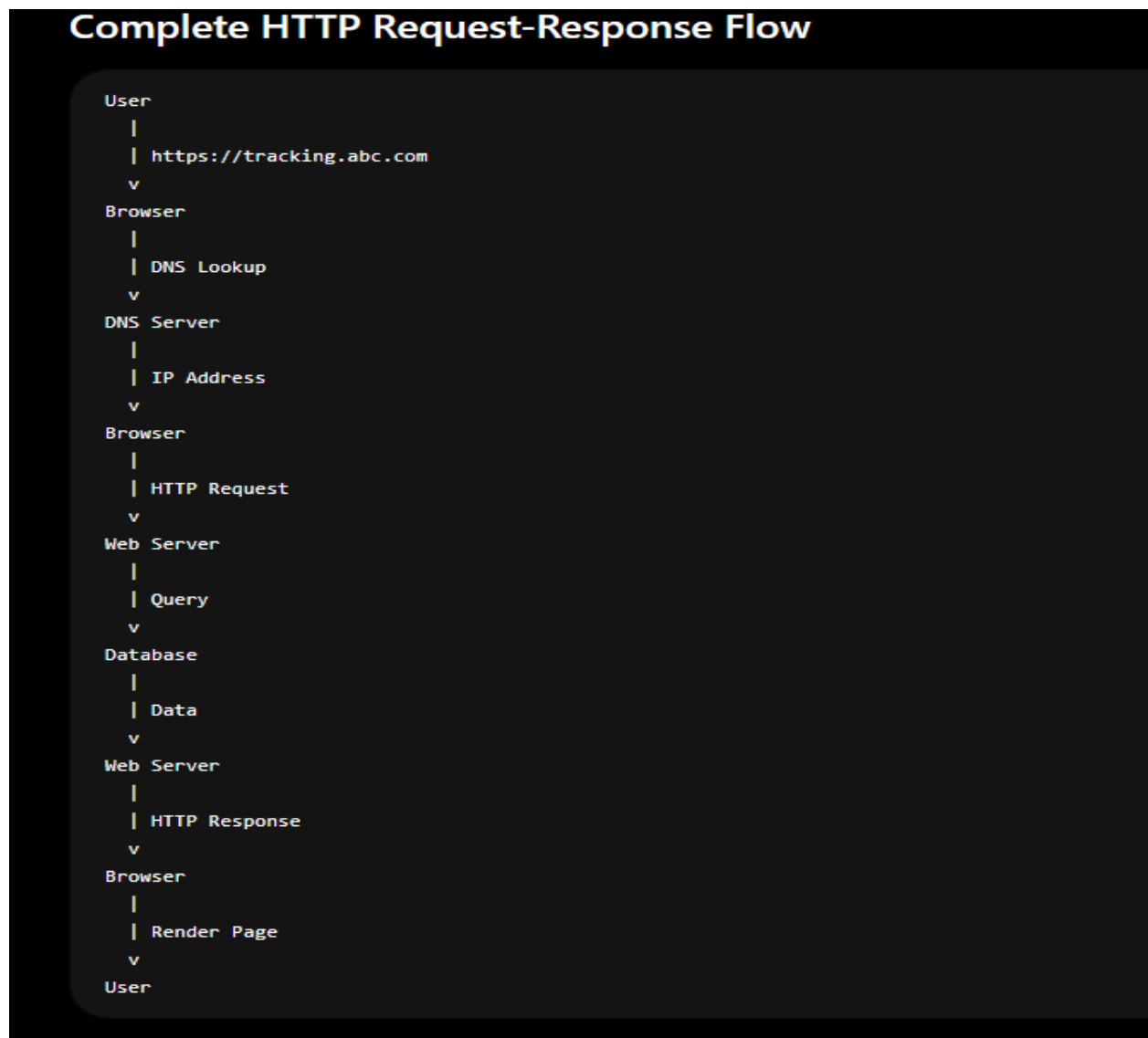
Step 5

Browser displays:

```
Shipment: ABC123
Status: Delivered
```

The Request-Response Cycle (HTTP Example)

1. Browser requests `https://example.com`.
2. DNS resolves domain to IP address.
3. Browser sends an HTTP GET request to the web server.
4. Web server processes request, fetches HTML, queries the database if needed.
5. Server sends an HTTP response with a status code (200 OK) and the webpage content.
6. Browser renders the webpage.



Synchronous vs Asynchronous Communication

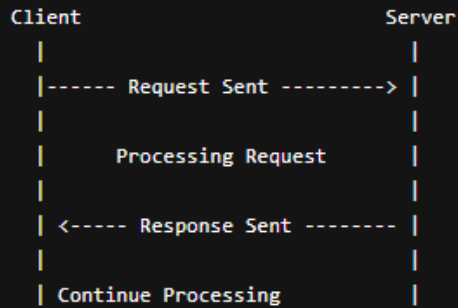
Synchronous Communication

- Client waits for a response before proceeding.
- Example: A user submits a form and waits for a confirmation message.
- Used in REST APIs, traditional web apps.

Asynchronous Communication

- Client doesn't wait for a response and can perform other tasks.
- Example: WebSockets for real-time chat, AJAX for background requests.
- Used in messaging apps, live updates, IoT applications.

Synchronous Communication Diagram



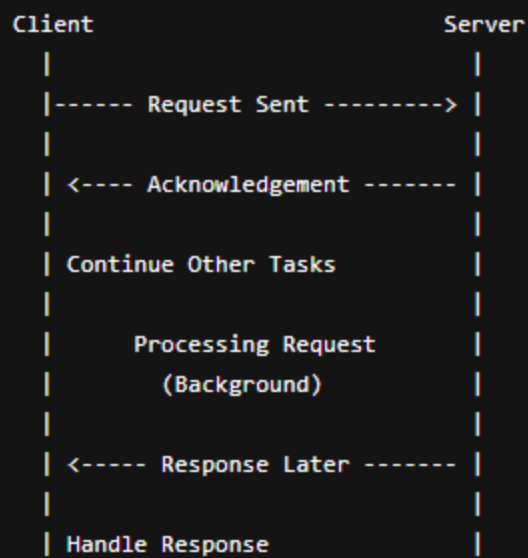
Key Point

Client waits until response is received

This is also called:

Blocking Communication

Asynchronous Communication Diagram



Key Point

Client continues other tasks

This is also called:

Non-Blocking Communication

Synchronous

```
Web App
|
| HTTP Request
v
Azure Function
|
| Process
v
Response
```

The caller waits.

Asynchronous

```
Web App
|
| Send Message
v
Service Bus Queue
|
v
Azure Function
|
v
Processing
```

The web application gets acknowledgment immediately and processing happens later.

This is exactly why queues improve scalability.

Stateless vs Stateful Servers

Stateless Servers

- No memory of past interactions; each request is independent.
- Example: REST APIs, HTTP servers.
- Benefits: Scalability, easy caching, load balancing.

Stateful Servers

- Maintain session information across requests.
- Example: WebSockets, multiplayer games, banking applications.
- Benefits: Personalization, seamless user experiences.

A stateless server does not retain client information between requests, while a stateful server maintains session or user state across multiple requests. Stateless architectures scale more easily, whereas stateful architectures provide richer session-based interactions.